

# Skills That Don't Exist: A Large-Scale Study of Hallucinated Skill Recommendation in LLM Agents

Weifeng Yuan

Huazhong University of Science and  
Technology  
Wuhan, China  
cougarsecu@gmail.com

Wenbo Guo✉

Nanyang Technological University  
Singapore  
honywenair@gmail.com

Feng Dong✉

Huazhong University of Science and  
Technology  
Wuhan, China  
dongfeng@hust.edu.cn

Haoyu Wang

Huazhong University of Science and  
Technology  
Wuhan, China  
haoyuwang@hust.edu.cn

Yang Liu

Nanyang Technological University  
Singapore  
yangliu@ntu.edu.sg

## Abstract

LLM agents dynamically acquire new capabilities by downloading *skills* from open registries. Instead of browsing these catalogs manually, developers typically ask the agent to recommend and install a skill for them. This convenience hides a risk: agents frequently invent names for skills that do not exist, confidently returning names that appear in no registry. We term this flaw *skill name hallucination*. A fake name may seem harmless, but it acts as an open door for supply-chain attacks. Because registries rarely verify publishers, an adversary can prompt the agent, collect the fake names it returns, and pre-register malicious skills under them, then wait for a victim to install the payload.

To uncover the severity of this threat, we conducted the first large-scale measurement of skill name hallucination. We evaluated 15,000 prompts across 12 configurations (4 standalone LLMs and 8 agents). To ensure absolute accuracy, we conservatively counted a name as hallucinated only if it was missing from all live registries and GitHub. The results reveal a systemic vulnerability, as every single configuration hallucinates. Hallucination rates average 36.0% for standalone LLMs and 36.9% for agents, rising to 43.1% when answering real-world developer questions. In total, the systems generated 5,669 distinct hallucinated names. Crucially, these names are not random noise. Agents predictably repeat the same fake names across different prompts and models, giving attackers highly reliable targets to hijack.

Finally, we tested four model-level defenses and found a severe conflict between security and usability. The strongest defense, retrieval grounding, successfully slashed the hallucination rate from 40.8% down to 3.2%. However, it crippled the agent's usefulness. Even the best-defended system could only recommend the correct skill about one out of every six times. Ultimately, skill name hallucination is a highly exploitable vulnerability requiring minimal attacker effort. Fixing it cannot rely on prompt engineering or model tuning alone. It demands ecosystem-wide structural changes, specifically registry-level name reservations and verified recommendation pipelines.

## CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; • **Software and its engineering** → **Software libraries and repositories**; • **Security and privacy** → **Malware and its mitigation**.

## 1 Introduction

To add a new capability to an AI coding agent, developers used to browse a registry. Most of the time they just ask the agent which skill to use, and install whatever it names. A *skill* is a small bundle of natural-language instructions, kept in a SKILL.md file, that teaches an agent to do one task. The Agent Skills ecosystem is new, but it has grown quickly. Anthropic published the Agent Skills standard in December 2025, and OpenAI adopted it within weeks [1, 2, 4, 5, 7, 8, 12, 17, 18, 33, 37–39]. Analysts expect agent-enabled enterprise applications to rise from under 5% in 2025 to 40% the following year [16]. The registries kept pace, and SkillsMP alone lists more than 1.7 million skills [48]. Nobody can read through a catalog that large, so the agent decides and the user installs what it suggests. This habit is visible online. On Stack Overflow, Reddit, and Hacker News, developers routinely ask for skill and tool recommendations [25, 27, 45].

This delegation rests on a quiet assumption. The skill the agent names should exist. Often it does not. Language models *hallucinate*, and they state invented facts with full confidence [23]. Inside an agent, this shows up as a recommended skill whose name matches no entry in any registry. We call this a *skill name hallucination*. This is not a rare error. On real user questions it is the common case, and it persists even when the agent has a web-search tool and is told to verify the name first. These hallucinated skill names are not random. The model falls back on capabilities and tools it already knows. Some are generic capability phrases, such as `file-system-operations` or `text-editor-tool`. Others borrow names of familiar tools and products, such as `visual-studio-code` or `ublock-origin`. A web search finds results for all of these, because they name real capabilities or tools. The search confirms that the name means something, but not that any published skill has that name. Web search runs the wrong check.

A hallucinated name is harmless only while nothing exists behind it. An attacker removes that condition in three steps. The

✉ Wenbo Guo and Feng Dong are corresponding authors.

attacker first queries public agents and records the names they invent again and again. The attacker then registers one of those names on an open registry, with malicious instructions hidden in the `SKILL.md`. This step is cheap, because these registries barely check who publishes. Later, a victim who asks a similar question receives the same name and installs it. Installation is not a real barrier. Many agents auto-approve it, and even when a prompt appears, users tend to approve a recommendation from their own agent [41]. The same agent that recommends the skill is the one that later runs it (Section 2.2). No existing supply-chain or injection defense stops the attack, because a hallucinated name gives a defender nothing to work with. There is no real name to match, and the payload can hide in plain language rather than code [20]. This is what separates the threat from package squatting [51], and Section 4.3 measures the resulting detection failure.

To measure how real this threat is, we run the first systematic study of skill name hallucination. We send 15,000 prompts to 12 evaluated configurations and check every name they recommend against the live registries. The prompts are of two kinds, community prompts drawn from developer forums and skill description prompts built from real published skills. The 12 evaluated configurations comprise 4 standalone LLM configurations and 8 agent configurations, most of them given a web-search tool. We organize the study around four questions that follow the attacker’s path:

- **RQ1 (Prevalence).** How often do agents hallucinate skill names, and how robust is that rate to the model, its temperature, its size, and the way users ask?
- **RQ2 (Targetability).** How much attack coverage is possible by registering recurring hallucinated skill names?
- **RQ3 (Detectability).** Why are hallucinated skill names hard to detect by the model itself and by edit-distance filters?
- **RQ4 (Mitigation).** How well do retrieval-augmented generation (RAG), self-refinement, and cross-model voting reduce hallucination, and at what cost to the agent’s usefulness?

Every evaluated configuration hallucinates, at rates from 6.5% to 62.0%. The surface is widest where it matters most. On synthetic prompts, where a correct skill is known to exist, the mean rate is 29.4%, but on real developer questions it rises to 43.1%. No model, temperature, size, or phrasing removes the problem, and a web-search tool does not either. The names are also easy to target. In the current live-validated persistence run, Claude Sonnet 4.6 repeats its dominant hallucinated skill name 7.8 times in 10. Most names are specific to one configuration, but 410 are shared by two or more configurations, and these shared names account for 15.3% of all hallucinated recommendation events. The same naming prior also reaches neighboring software ecosystems, where 851 hallucinated skill names are real PyPI or npm packages.

The names are also hard to detect. A model asked to verify a name accepts real and hallucinated skill names at nearly the same rate, yielding 51% balanced accuracy, no better than a coin flip. They sit a median of six edits from any real skill, far beyond the reach of typosquatting filters. Defenses narrow the surface but do not close it. RAG cuts confirmed hallucination by 37.6 percentage points on average, from 40.8% to 3.2%, while self-refinement proves far less effective from 40.8% to 33.2%. Even the best-defended agent names

the right skill only about one time in six, so it is safe mostly because it returns no output. The ecosystem stays hard to use safely.

We make the following contributions:

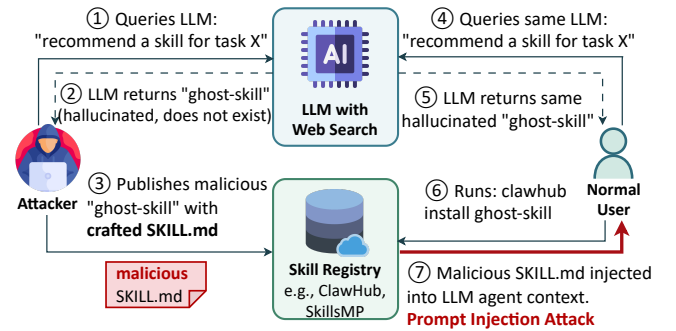
- **A new attack on the agent skill supply chain.** We identify skill name hallucination as the entry point of a skill name squatting attack, in which an agent’s own persistent hallucinations let an attacker pre-register the names that the agent will later recommend, and we show why this falls outside the reach of existing supply-chain and prompt-injection defenses (Section 2.2).
- **A measurement of the attack surface and its targetability.** Across 15,000 prompts and 12 evaluated configurations, we measure how prevalent, how reusable, and how stealthy hallucinated skill names are, including a new skill name squatting coverage analysis and a cross-ecosystem overlap between skills and Python and npm packages (Section 4).
- **An evaluation of defenses and their cost.** We test four defenses and report both their effect on the attack surface and their cost to the agent’s usefulness (Section 4.4).

## 2 Background

### 2.1 Skill Name Hallucination

Agents load skills progressively. The agent first sees only a skill’s name and description, and reads the full `SKILL.md` instructions only after it selects the skill. The name is therefore both a discovery signal and a security-sensitive input. Skill selection fails when the agent names a skill that does not exist. We call such a recommendation a *skill name hallucination*. The failure is similar to a known one in code generation, where models recommend packages that do not exist on PyPI or npm [40, 51].

### 2.2 Threat Model



**Figure 1: Overview of the skill hallucination and prompt injection attack.**

**Participants.** We model four parties: the *agent*, which recommends and later loads a skill; the *registry*, such as ClawHub or SkillsMP, which accepts community skills; the *victim*, who installs the returned name; and the *attacker*, an ordinary user of the same public agents and registries. The attack relies on misplaced trust: the victim trusts the agent, the registry trusts publishers, and the agent cannot distinguish a real skill from one it invented (Section 4.3). Unlike package hallucination, one agent can invent, recommend, and later execute the installed skill.

**Attack flow.** The attack has three steps (Figure 1). First, the attacker queries public agents and records skill names that recur. Second, the attacker publishes a skill under one such name, with malicious instructions or scripts in its SKILL.md package. This requires little privilege: ClawHub accepts week-old accounts, and SkillsMP indexes public repositories. Third, a later victim receives the same hallucinated name and installs it. Even when installation is not automatic, prompts often show only the name, not the hidden instructions.

### 3 Experiment Design

To measure skill name hallucination across models and agents, we use a three-stage pipeline. (1) *Prompt Construction* builds a dataset of skill-seeking prompts. (2) *Recommendation Generation* sends each prompt to every model and agent and records the skill names they return. (3) *Skill Name Validation* checks each returned name against the live registries and counts any name no registry contains as a hallucination.

#### 3.1 Prompt Construction

Our dataset has 15,000 prompts of two types, each measuring a different aspect of hallucination. *Community Prompts* are real questions drawn from developer forums. They measure how often an agent hallucinates on the questions users actually ask. Many of these questions have no matching skill, so the model must either state that none exists or invent a name. *Skill Description Prompts* are built from the description field of a real published skill’s SKILL.md, the short text that states what the skill does. Because each prompt comes from a real skill, a correct skill is known to exist for every one. These prompts measure whether a model can output a correct skill name it should be able to find. Together, the two types separate hallucination caused by a missing skill from hallucination that remains even when the right skill exists.

**3.1.1 Community Prompts.** To make the hallucination rate reflect a real attack surface rather than an artificial one, we collect prompts that match the questions real users ask. We mine real questions from three developer communities: Stack Exchange technical sites, AI and developer subreddits, and Hacker News (retrieved through the Algolia API). In each source, we search for skill-seeking questions with queries such as “what tool can I use,” “best plugin,” and similar skill recommendation phrases.

Most forum posts are irrelevant to our task, so before building prompts we score each post with a set of relevance rules. The rules use the post title, body, and source community. They add evidence for a software skill question when a post contains a target term (*skill*, *tool*, *plugin*, *extension*, *mcp*, *agent*), software or AI context (e.g., *github*, *codex*, *claude*, *llm*), a technical community (e.g., Stack Overflow, *r/vscode*), or a question form. They subtract evidence when a post uses *skill* in the human sense (e.g., *soft skills*, *cooking skills*), comes from a nontechnical community (e.g., *r/AskReddit*), or mentions a target term without technical context, which usually refers to a physical tool or human ability. We keep only posts whose score exceeds a threshold, so genuine software skill questions pass while irrelevant posts fall below the cutoff. The result is 5,000 community prompts covering skill name inquiries (2,395), skill

recommendations (1,842), and usage descriptions (763), drawn from Stack Exchange (2,484), Reddit (1,878), and Hacker News (638).

**3.1.2 Skill Description Prompts.** Community prompts show how often agents hallucinate in the wild, but for many of them no real skill fits. We therefore cannot tell whether a hallucination comes from a missing skill or happens even when a correct skill exists. Skill description prompts resolve this ambiguity, because each one is built from a real published skill, so a correct answer is known to exist. We take the 10,000 most popular skills on *SkillsMP* and turn each skill’s description field into one prompt that asks the model to return the matching skill’s name. Because each prompt comes from a real skill, a correct answer always exists, so any hallucination here cannot be attributed to a missing skill.

#### 3.2 Recommendation Generation

We evaluate skill name hallucination on two kinds of system, standalone LLMs and LLM-powered agents. The main design challenge is the knowledge cutoff. Anthropic released the Agent Skills standard on December 18, 2025 [4], after the training cutoff of most models we test. These models could then hallucinate because they never saw a real skill during training, not because of any deeper tendency. To rule out this explanation, we give every pre-cutoff model and agent a web-search tool [36], require it to search before it recommends a skill, and confirm that search works for each one. The one exception is Claude Opus 4.7 (cutoff January 2026), which already knows the standard. We run it without search to approximate the best case for training-time knowledge.

Table 1 lists the models and agents we evaluate. We use four standalone LLM configurations and eight agent configurations spanning four agent frameworks. Claude Code and Codex search the web natively, while OpenClaw and OpenCode use a self-hosted SearXNG instance through MCP. We run OpenCode with five backends (GPT-5.4-mini and four open-source models). Together, the 4 standalone LLM configurations and 8 agent configurations give the 12 configurations we report throughout the paper. Before running the skill prompts, we checked that web search worked for every search-enabled system. We asked each one to fetch a current-news item on the test day. This check only confirmed that search was reachable, and we excluded it from the hallucination measurements.

**Table 1: Evaluated configurations**

| System            | Type  | Model           | Knowledge   | Web Search    |
|-------------------|-------|-----------------|-------------|---------------|
| GLM-4.5-Air       | LLM   | glm-4.5-air     | Pre-release | Enabled       |
| GPT-5.4-mini      | LLM   | gpt-5.4-mini    | Pre-release | Enabled       |
| Claude Sonnet 4.6 | LLM   | sonnet-4-6      | Pre-release | Enabled       |
| Claude Opus 4.7   | LLM   | opus-4-7        | Jan 2026    | Disabled      |
| Claude Code       | Agent | sonnet-4-6      | Pre-release | Enabled       |
| Codex             | Agent | gpt-5.3-codex   | Pre-release | Enabled       |
| OpenClaw          | Agent | gpt-5.4-mini    | Pre-release | SearXNG (MCP) |
| OpenCode          | Agent | gpt-5.4-mini    | Pre-release | SearXNG (MCP) |
| OpenCode          | Agent | 4 Ollama Models | Pre-release | SearXNG (MCP) |

The OpenCode row with four Ollama models expands to four agent configurations: Ministral-3 8B, Granite 3.3 8B, Qwen 3.5 9B, and Nemotron-Cascade-2 30B. Later figures shorten these names to Min3-8B, Gra3.3-8B, Qwen3.5-9B, and Nem2-30B.

Figure 2 shows the two prompt templates. For *community prompts*, the system message instructs the model to act as a skill recommender and requires kebab-case names. The user message adds the

forum post, the required output format, and an instruction to search the web first. For *skill description prompts*, the system message asks the model to identify the one real skill that matches, and to return an empty `skill_names` list rather than guess when it is unsure. Both templates require a JSON reply of the form `{"skill_names": [...], "answer": "..."}.` We cap each reply at 1,200 tokens and leave all sampling parameters at provider defaults, except in RQ2, where we vary them on purpose.

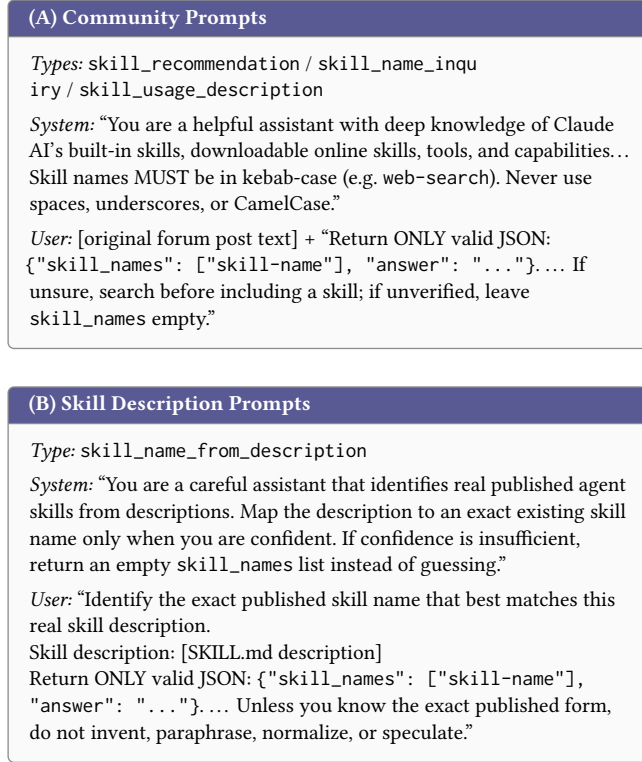


Figure 2: Prompt templates.

### 3.3 Skill Name Validation

We treat a recommended skill as a hallucination when it exists on no skill-hosting registry and in no open-source repository. To verify whether a skill is real, we use a two-stage validation. **Registry verification.** We match the name against a broad reference index built from multiple public sources of real skill names, including the SkillsMP and ClawHub registries, the curated awesome-openclaw list, Claude and Codex skill repositories, historical `SKILL.md` records, and our prior registry lookups. SkillsMP in particular is built by periodically harvesting skill from public GitHub repositories and now indexes over 1.7 million skills, so a name missing even from this large catalog is already very likely fake. **Expanded verification.** For every name the registries do not resolve, we expand the search to GitHub. We query GitHub Code Search with `"name: <canonical-name>" filename: SKILL.md`, fetch the candidate `SKILL.md` files, and accept a match only when a candidate’s front-matter name:

field canonicalizes to the generated name. This stage is slow and costly, but it ensures that a real skill too new to appear in the registries is never miscounted. We label a name a hallucination only when both stages fail.

We take the names from the `skill_names` field of each response and match them against this validation pipeline. We first normalize both generated and reference names by trimming wrappers and markup, splitting camel case, converting `&` to `and`, replacing non-alphanumeric runs with hyphens, and lowercasing. A name counts as real on an exact match, a unique canonical-alias match, or a matching name: field in a `SKILL.md` found by either stage. We discard clear extraction artifacts, such as URLs, paths, schema tokens, trace IDs, and sentence-like strings. None of these can be a skill name, which is a short kebab-case identifier, so they are noise from parsing the response rather than genuine recommendations. We exclude them instead of counting them as hallucinations. We cache all lookups to avoid repeated searches.

Our primary metric is the *skill name hallucination rate*. We compute this metric at the unique-name level, not over repeated recommendation events. Let  $R$  be the set of distinct canonical names a system recommends, after we discard the extraction artifacts above. For each name  $r \in R$ , the indicator  $\mathbb{1}[r \text{ is hallucinated}]$  equals 1 when  $r$  fails both verification stages and 0 otherwise. The hallucination rate is the share of these unique recommended names that are hallucinated:

$$\text{Hallucination Rate} = \frac{\sum_{r \in R} \mathbb{1}[r \text{ is hallucinated}]}{|R|} \quad (1)$$

A higher rate means a wider attack surface, because more of an agent’s recommendations point to skills that do not exist. On the skill description prompts, where a correct skill is known to exist, we additionally report *recall*, the fraction of prompts for which the model returns the correct name.

## 4 Empirical Results

We investigate four research questions, each on a distinct dimension of skill name hallucination. RQ1 measures prevalence, how often agents hallucinate and whether the rate holds across model family, temperature, size, and prompt domain. RQ2 measures targetability, how often hallucinated names repeat, how widely they are shared across configurations, how often they overlap with real software packages, and how few names cover most cases. RQ3 measures detectability, whether self-auditing or lexical distance to real skill names can identify hallucinated names. RQ4 measures mitigation, how well RAG, self-refinement, and cross-model voting reduce hallucination.

### 4.1 RQ1: Prevalence

To systematically measure the prevalence of this attack surface, we evaluated all 12 configurations on the full 15,000-prompt corpus. Every recommended name was validated using the two-stage pipeline from Section 3.3. To avoid duplicate counting, hallucination rates are computed over unique skill names rather than repeated recommendation events. Table 2 reports these statistics, separating name-level metrics and prompt-level outcomes to illustrate the

varying attack surfaces under different external grounding methods. The results demonstrate that hallucination is ubiquitous, with rates ranging from 6.5% for GPT-5.4-mini to 62.0% for OpenCode with Ministral-3 8B, averaging 36.0% across standalone LLMs and 36.9% across agents.

**Table 2: Per-configuration hallucination statistics**

| Model                           | Search   | Unique | Matched | Halluc. | Other | H. Rate | No output |
|---------------------------------|----------|--------|---------|---------|-------|---------|-----------|
| <i>LLM direct API</i>           |          |        |         |         |       |         |           |
| Claude Sonnet 4.6               | Enabled  | 6,039  | 2,970   | 2,982   | 87    | 49.4%   | 26.6%     |
| GPT-5.4-mini                    | Enabled  | 5,602  | 5,207   | 363     | 32    | 6.5%    | 52.5%     |
| GLM-4.5-Air                     | Enabled  | 5,771  | 2,695   | 2,965   | 111   | 51.4%   | 46.0%     |
| Claude Opus 4.7                 | Disabled | 2,051  | 1,292   | 753     | 6     | 36.7%   | 56.8%     |
| <i>Coder Agent</i>              |          |        |         |         |       |         |           |
| Claude Code Sonnet 4.6          | Enabled  | 234    | 153     | 81      | 0     | 34.6%   | 90.9%     |
| OpenClaw GPT-5.4-mini           | SearXNG  | 4,677  | 3,355   | 1,193   | 129   | 25.5%   | 56.1%     |
| OpenCode GPT-5.4-mini           | SearXNG  | 1,231  | 898     | 287     | 46    | 23.3%   | 92.2%     |
| Codex GPT-5.3-Codex             | Enabled  | 3,770  | 2,601   | 1,100   | 69    | 29.2%   | 76.2%     |
| OpenCode Ministral-3 8B         | SearXNG  | 4,830  | 1,276   | 2,994   | 560   | 62.0%   | 81.8%     |
| OpenCode Granite 3.3 8B         | SearXNG  | 4,874  | 2,027   | 2,693   | 154   | 55.3%   | 69.0%     |
| OpenCode Qwen 3.5 9B            | SearXNG  | 4,551  | 2,716   | 1,719   | 116   | 37.8%   | 80.0%     |
| OpenCode Nemotron-Cascade-2 30B | SearXNG  | 4,389  | 3,094   | 1,208   | 87    | 27.5%   | 69.3%     |

Unique equals Matched, Halluc., and Other, which represent verified real skills, unverified hallucinations, and extraction artifacts, respectively. H. Rate is the ratio of Halluc. to Unique. No output indicates the fraction of prompts yielding no skill recommendation.

Rather than generating arbitrary noise, the models predominantly output semantically plausible but non-existent skill identifiers. Qualitative analysis reveals two dominant categories. The first comprises abstract capability descriptions (e.g., file-system-operations or text-editor-tool), where the model attempts to describe the required action instead of recalling a verified skill. The second involves adopting the names of real-world software or ecosystem objects that have not been officially registered as skills (e.g., visual-studio-code or ublock-origin).

External grounding via web search fails to mitigate this vulnerability. Despite enabling three of the four standalone LLMs with built-in search tools and explicitly prompting them to verify names prior to output, hallucination rates remain critically high (e.g., 49.4% for Claude Sonnet 4.6 and 51.4% for GLM-4.5-Air). This failure stems from the semantic plausibility of the hallucinated names, because searching for file-system-operations returns abundant relevant results, thereby tricking the agent into confirming the concept’s existence rather than checking registry status.

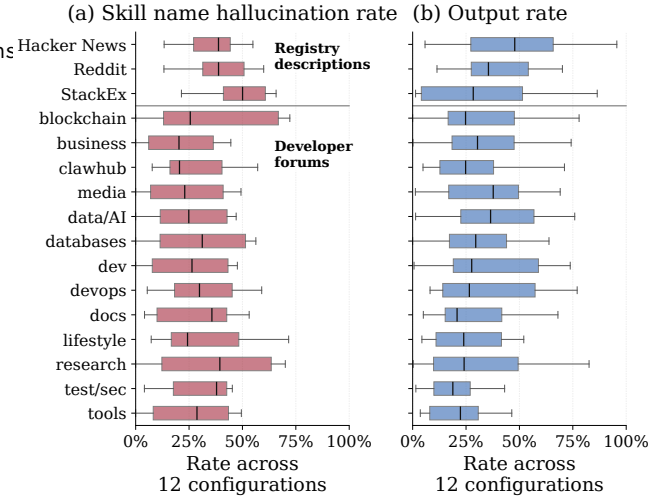
Furthermore, conservative output generation artificially deflates hallucination rates without addressing the underlying threat. For instance, GPT-5.4-mini achieves a deceptively low rate of 6.5% primarily because it returns no skill on 52.5% of the prompts. This behavior is more extreme in agent configurations, with Claude Code returning no output for 90.9% of prompts and yielding the fewest names overall, yet 34.6% of its outputs are still hallucinated. Consequently, higher no-output rates simply limit output volume, leaving the output skill names highly vulnerable to hallucination.

**Finding 1.** Skill name hallucination is ubiquitous and persists despite basic safeguards. All 12 configurations hallucinate skill names, with rates from 6.5% to 62.0%. The root cause is that models often turn a capability into a name, such as file-system-operations or text-editor-tool, instead of recalling a published skill. Furthermore, two system behaviors fail to fix this. Web search can confirm that such phrases exist, but not that they are valid skill names. Finally, more

conservative output reduces the volume of recommended names, but the names that remain are often hallucinated.

The evaluation of Claude Opus 4.7 provides insights into whether recent training data inherently neutralizes the attack surface. As its January 2026 knowledge cutoff postdates the December 2025 Agent Skills release [6], this configuration isolates the impact of training-time knowledge updates. Operating without web search, the model adopts a highly restrictive generation strategy, returning no skills for 56.8% of prompts. Although its hallucination rate drops to 36.7%, falling below that of Claude Sonnet 4.6, it remains higher than several search-enabled agents. For example, OpenCode with GPT-5.4-mini and OpenClaw with GPT-5.4-mini achieve lower rates of 23.3%, 25.5%, respectively. Thus, updated training corpora reduce output volume but do not remove the attack surface.

This vulnerability is particularly pronounced under real-world usage conditions. As Figure 3 shows, forum queries pose a consistently higher risk than synthesized skill description prompts. The median hallucination rate reaches 38.8% on Hacker News and Reddit, and peaks at 50.1% on Stack Exchange. Because forum questions closely resemble the queries an attacker’s victim would issue in practice, this gap expands the real-world attack surface.



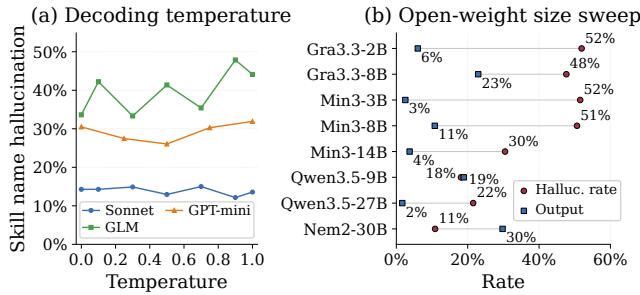
**Figure 3: Hallucination rate by prompt source.**

To assess whether this attack surface can be mitigated through hyperparameter tuning or model scaling, we evaluated various decoding temperatures and parameter sizes using prompt subsets of 4,000 prompts. Figure 4(a) shows that changing the temperature for search-enabled LLMs from 0 to 1.0 produces little improvement. The per-prompt traces show surface-level variation without recovery. For the Retell AI troubleshooting skill, whose real name is retell ai-common-errors, GLM-4.5 produced retellai-troubleshooting, retellai-troubleshooter, fix-retell-ai, and retellai-debug across temperatures, but never the real name. Similarly, for the CPU monitoring skill monitoring-cpu-usage, the same model alternated between cpu-performance-monitor and cpu-performance-analyzer. These examples illustrate that temperature mainly



changes which plausible paraphrase is emitted, rather than forcing the model to verify the registered identifier.

Figure 4(b) shows a similar pattern for model scaling across Granite 3.3, Ministral-3, Qwen 3.5, and Nemotron-Cascade-2. Larger variants sometimes reduce the aggregate rate, such as the drop in Ministral-3 from 51% at 8B to 30% at 14B, but they still produce semantically close false names. In the Ministral-3 14B run, the model mapped `k8s-deploy` to `kubernetes-progressive-delivery` and mapped `rust-skill-creator` to both `crate-skill-generator` and `rustdoc-skill-generator`. Qwen 3.5 27B similarly mapped `exa-incident-runbook` to `execute-exa-incident-response-procedures`, `evidence-auditor` to `audit-claims-evidence`, and `openevidence-enterprise-rbac` to `openevidence-enterprise`. Even the largest evaluated model, Nemotron-Cascade-2 30B, retained an 11% hallucination rate over emitted names, corresponding to 390 hallucinated names among 3,588 recommendations on the shared 4,000-prompt subset. Thus, while scaling reduces the frequency of hallucinations across some model families, it fails to eliminate the underlying mechanism, as models continue to translate descriptions into plausible aliases similar to skills rather than recovering the exact published names.



**Figure 4: Hallucination rates across decoding temperatures and model sizes.**

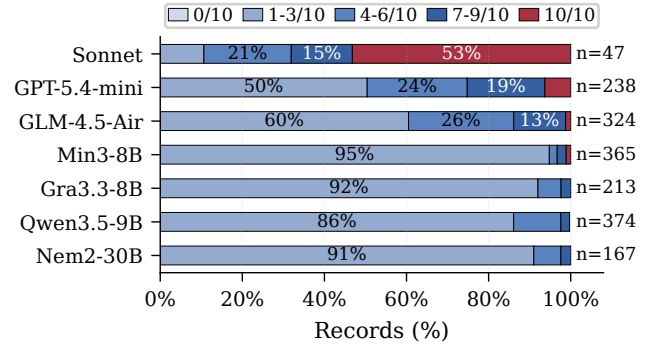
Finding 2. The attack surface is structural, not a setting. No evaluated model, temperature, or size setting removes the problem. Search can mask it, temperature mostly changes which hallucinated skill name appears, and larger models can still hallucinate whenever they answer. The issue is built into the task, not one special configuration.

## 4.2 RQ2: Targetability

The exploitability of this attack surface depends on predictability. To successfully execute a skill name squatting attack ahead of time, adversaries must anticipate which hallucinated names agents will generate before victims query them. Consequently, a hallucinated name becomes a high value target under two conditions: (1) high intra-model persistence, where a specific model reliably repeats the name to ensure consistent traps for victims, and (2) high inter-model convergence, where multiple models independently generate the same name to maximize the impact scope of a single registered payload. We evaluate both dimensions to quantify the targeting budget required for an attacker to achieve broad victim coverage.

Our experiment reveals high intra-model determinism for hallucinated names. By querying seven configurations 10 times each

with 500 random prompts from RQ1, we define the dominant hallucination for each configuration and prompt as the confirmed hallucinated name that appears most often across the 10 runs, and then track how often it recurs. As shown in Figure 5, Claude Sonnet 4.6 exhibits striking persistence, repeating its dominant hallucination 7.8 times out of 10 on average, and returning the exact same name across all 10 runs in 53% of cases. This stickiness stems from the model’s adherence to instructions rather than local caching. Across all 10 trials, it generated citation-compass instead of the real citation-chasing-mapping, ideogram-monitoring-observability instead of ideogram-observability, and evernote-debug-bund instead of evernote-debug-bundle. While open-weight models are more volatile, repeating 1.5 to 2.0 times, highly persistent models allow attackers to reliably predict future outputs based on a single observed recommendation.



**Figure 5: How often a model repeats hallucinated skill name across 10 runs.**

Beyond intra-model persistence, hallucinated names demonstrate significant inter-model convergence. Among the 1,728 prompt and model pairs with confirmed hallucinations, 219 dominant names were generated by at least two distinct models. For example, six independent models produced `classification-model-builder`, and five converged on names like `exa-incident-response`, `firebase-genkit` and `debugging-guidelines`. This indicates that models share underlying naming heuristics. When presented with a plausible capability, they predictably converge on identical hallucinated names. For an attacker, this shared behavior means a single hallucinated name can compromise users across entirely different LLM ecosystems.

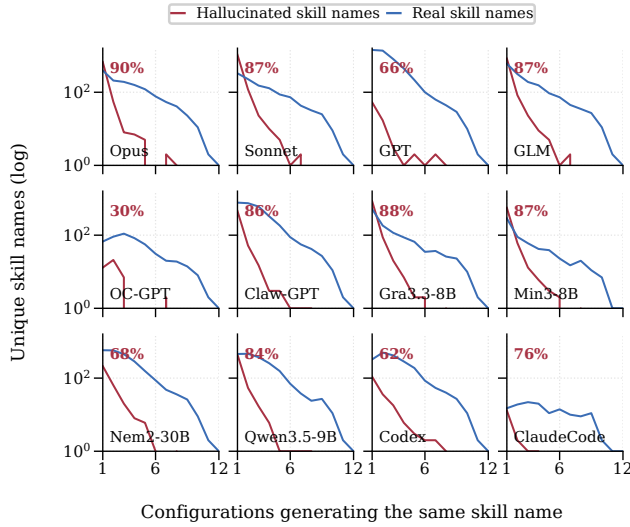
A measurable sign of this shared naming prior is overlap with the broader developer software ecosystem. To test whether hallucinated skill names reflect neighboring package namespaces, we queried PyPI and npm for the 5,669 unique confirmed hallucinated names. As shown in Table 3, 851 names, or 15.0%, exactly match an existing package, even though the prompts requested agent skills rather than software packages. These matches are not valid skills. They indicate that models sometimes reuse familiar package or tool identifiers as names for skills. The overlap is concentrated in single configurations, but 126 names recur across two or more configurations, supporting the hypothesis that neighboring ecosystems shape predictable cross-model names.

**Table 3: Hallucinated skill names that match a PyPI or npm package**

| #Configs     | Records      | Unique names | % of total*  | PyPI / npm       |
|--------------|--------------|--------------|--------------|------------------|
| 1            | 725          | 725          | 12.8%        | 349 / 376        |
| 2            | 170          | 85           | 1.5%         | 44 / 41          |
| 3            | 90           | 30           | 0.5%         | 19 / 11          |
| 4            | 32           | 8            | 0.1%         | 5 / 3            |
| 5            | 5            | 1            | <0.1%        | 0 / 1            |
| 6            | 6            | 1            | <0.1%        | 0 / 1            |
| 8            | 8            | 1            | <0.1%        | 1 / 0            |
| <b>Total</b> | <b>1,036</b> | <b>851</b>   | <b>15.0%</b> | <b>418 / 433</b> |

\*Percentage out of 5,669 unique, confirmed hallucinated names.

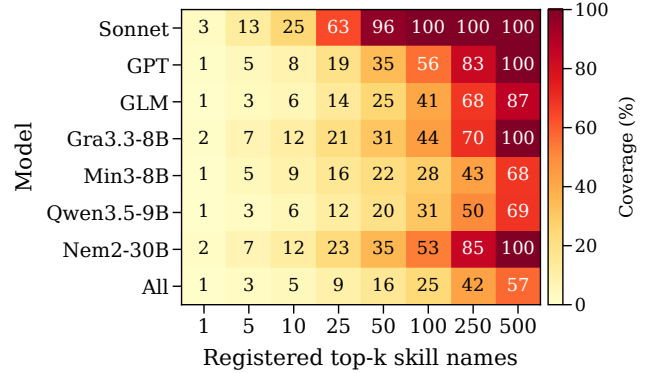
While the absolute number of unique hallucinated names is vast and highly fragmented across configurations, the actual event frequency is heavily concentrated in shared names. Figure 6 illustrates the full set of names recovered from the 15,000-prompt corpus. Counted purely by distinct identifiers, 92.8% of hallucinated names are exclusive to a single configuration. However, weighting these names by recommendation frequency reveals a different threat landscape. The 410 names that appear in at least two configurations account for 15.3% of all 6,212 hallucination events, which is double their 7.2% share by distinct count. Thus, most hallucinated names appear only once, but the names that recur are recommended often enough to become highly reusable targets.



**Figure 6: Cross-configuration reuse of hallucinated and real skill names.**

This concentration allows attackers to achieve high impact with a minimal targeting budget. Figure 7 ranks hallucinated names by recurrence frequency to calculate the coverage an attacker would achieve by squatting the top- $k$  names. Each cell reports the percentage of hallucinated recommendations covered at that budget, and the “All” row pools events across the seven configurations in the persistence experiment. For Claude Sonnet, registering merely

the top 25 names intercepts 63% of all its hallucinated recommendations, and 50 names cover 96%. Even aggregated across all seven configurations, squatting the top 500 names covers 57% of all hallucinated recommendations. An attacker does not need to predict every rare hallucination, and a cheaply harvested dictionary of a few hundred predictable names sits exactly in the execution path of a majority of victims.



**Figure 7: Coverage from registering top-ranked hallucinated skill names.**

**Finding 3:** Hallucinations exhibit deterministic and cross-model patterns, enabling a highly efficient ahead-of-time skill name squatting attack. Rather than generating random noise, models draw from shared naming priors as they reliably repeat specific hallucinated names with a frequency of up to 7.8 out of 10 times, independently converge on identical names across different LLM families, and overlap with PyPI or npm package names for 15.0% of unique hallucinated skill names. Due to this concentration, an attacker who squats the top 500 harvested names would cover 57% of hallucinated recommendation events in our seven-configuration persistence experiment.

#### Case study: binding repeated names to installable skills.

To verify the registration and installation path for predictable targets, we ran a benign proof-of-concept in an isolated Claude Code sandbox. We first selected `classification-model-builder`, a name our persistence experiment proved Claude Code reliably hallucinates for a specific prompt. We ethically created a public GitHub repository, added a benign `SKILL.md` file matching this name, and published a v1.0.0 release. We then fed the original targeted prompt to Claude Code. As expected, the agent recommended the hallucinated skill. We then instructed the agent to install this recommendation with a GitHub access token enabled, the agent automatically searched the platform, found our repository, and successfully installed the placeholder skill. This confirms the delivery channel without using a malicious payload. Once an attacker registers a persistent hallucinated name, the agent workflow can turn a model mistake into an installed attacker-controlled skill.

### 4.3 RQ3: Detectability

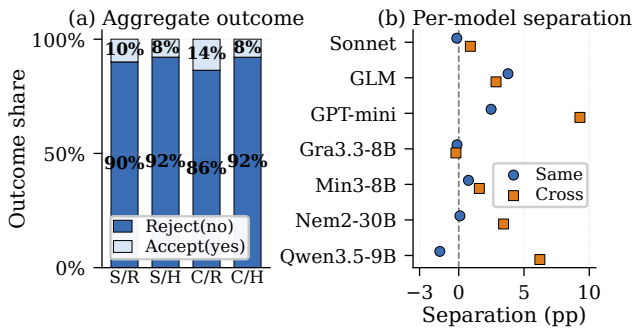
While RQ1 and RQ2 demonstrate the prevalence and predictability of this attack surface, an effective defense depends on whether

defenders can identify hallucinated names prior to execution. We systematically evaluate two detection strategies: LLM self-auditing and typosquatting filters based on edit-distance analysis.

We first evaluate whether models can identify hallucinations in generated skill names. We asked seven models a yes/no question to check if a given skill name exists. Each model judged real skills against confirmed hallucinations under two conditions. The first is same-model testing for its own hallucinations. The second is cross-model testing for hallucinations pooled from the other six models. After removing unclear non-binary replies, we retain 3,707 of 4,448 model responses as valid binary judgments.

Figure 8 illustrates why this defense is fundamentally ineffective. As shown in Figure 8(a), the models overwhelmingly default to “no” and reject almost everything, specifically 90.6% of names in the same-model condition and 88.8% in the cross-model condition. While this extreme bias safely blocks 91% of hallucinations, it destroys the agent’s usefulness. The models recognize only 10.0% of real skills among their own outputs and 13.6% among those of other models. Even when a model does answer “yes” to its own generated names, it is still wrong 22.7% of the time. Consequently, the per-model separation shown in Figure 8(b) yields an aggregate balanced accuracy of merely 51% for same-model tests and 53% for cross-model tests. Averaged across the two classes, the performance is no better than guessing.

The same-model and cross-model comparison helps explain this failure. A model performs worse at judging its own hallucinations than those of its peers because it uses the same internal logic to review the name as it did to generate it. GPT-5.4-mini is the only partial exception. It shows a less extreme rejection bias of 72.7% and the highest real-skill recall of 28.0%, yet it still fails to audit reliably. Ultimately, a model cannot filter a mistake it is built to make.

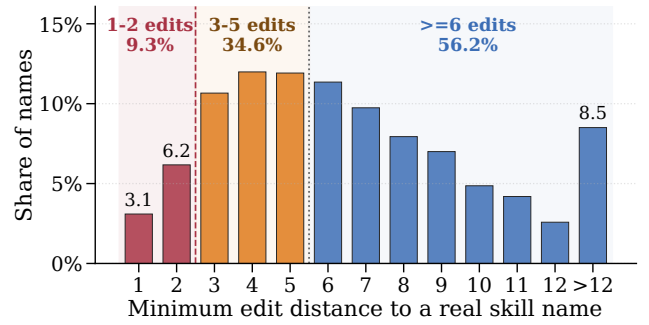


**Figure 8: LLM self-detection.** (S/C: same-/cross-model, R/H: real/hallucinated)

**Finding 4.** A model cannot audit an error it is built to make. Asked to distinguish real skill names from hallucinated ones, a model is no better than a coin flip, with 51% balanced accuracy on same-model tests and 53% on cross-model tests. Same-model auditing also has lower overall accuracy than cross-model testing, 32.2% vs 46.6%, because models reject many real skills along with hallucinated ones. The self-auditing

step inherits the same naming prior that produced the mistake in the first place, so it cannot reliably catch the flaw.

Traditional typosquatting defenses are inadequate because hallucinated names are novel semantic constructs rather than minor misspellings. Standard typosquatting filters typically flag variations within a Levenshtein distance of one or two from known real skills. However, comparing 5,669 confirmed hallucinated names against a catalog of 22,805 real skills reveals a mean edit distance of 6.85, as shown in Figure 9. While 9.3% of hallucinations represent near-misses, such as single substitutions, that such filters can catch, the vast majority, 56.1% at  $\geq 6$  edits, bypass them entirely. These distant outliers are not typos, but rather long, newly synthesized capability descriptions, rendering structural similarity checks useless.



**Figure 9: Distribution of Levenshtein distances of hallucinated/real skill names.**

**Finding 5.** Hallucinated names are new phrases rather than typos. With a mean distance of 6.85 edits and a median of 6, hallucinated names are usually too far from any real skill for a typosquatting filter to catch. Only 9.3% of the names sit close enough to look like ordinary misspellings. Therefore, edit-distance filters are largely ineffective. They catch only the few errors that resemble simple typos and completely miss the vast majority of newly invented names.

#### 4.4 RQ4: Mitigation

Finally, we investigate whether defenders can effectively mitigate this attack surface and evaluate the corresponding trade-offs regarding agent utility. To this end, we systematically evaluate four defensive strategies. Three operate within a single-model context, including RAG, self-refinement, and their combination. The fourth is a cross-model defense that requires consensus among multiple independent models before accepting a generated name. We applied the single-model defenses to the four open-weight models from RQ1 using a standardized subset of 2,000 prompts. Table 4 summarizes the resulting hallucination and no-output rates, where no output denotes cases producing no skill recommendation.

We first evaluate whether RAG can neutralize the attack surface by grounding models in real skill names. Using RAG [28], we build a vector index over real skill names and their available descriptions with all-MiniLM-L6-v2 sentence embeddings and the Chroma



**Table 4: Skill name hallucination rate and no-output rate**

| Model                  | Baseline |           | RAG   |           | Self-refinement |           | RAG + self-refinement |           |
|------------------------|----------|-----------|-------|-----------|-----------------|-----------|-----------------------|-----------|
|                        | Hall.    | No output | Hall. | No output | Hall.           | No output | Hall.                 | No output |
| Ministral-3 8B         | 61.3%    | 81.9%     | 1.6%  | 74.2%     | 65.0%           | 99.3%     | 0.4%                  | 86.9%     |
| Granite 3.3 8B         | 46.4%    | 70.2%     | 10.0% | 63.0%     | 38.3%           | 56.1%     | 9.8%                  | 69.7%     |
| Qwen 3.5 9B            | 32.5%    | 77.0%     | 0.9%  | 76.2%     | 11.4%           | 97.5%     | 0.7%                  | 88.5%     |
| Nemotron-Cascade-2 30B | 22.8%    | 66.5%     | 0.6%  | 58.4%     | 18.0%           | 96.7%     | 0.1%                  | 72.5%     |
| Mean                   | 40.8%    | 73.9%     | 3.2%  | 68.0%     | 33.2%           | 87.4%     | 2.7%                  | 79.4%     |

backend. For each user query, we embed the original prompt, retrieve the ten nearest skills by semantic similarity, format their names as a plain-text reference list, and prepend this list to the query under a system instruction that permits recommendations only from the retrieved names. We use ten retrieved skills to provide enough candidate coverage while keeping the context short. As shown in Table 4, this defense yields the largest drop in hallucinations. For example, Qwen 3.5 9B falls from 32.5% to 0.9%, and Ministral-3 8B from 61.3% to 1.6%. On average, RAG cuts the hallucination rate by 37.6 percentage points. However, this defense is not free. It requires index maintenance, adds token costs, and creates retrieval-poisoning exposure [11, 58].

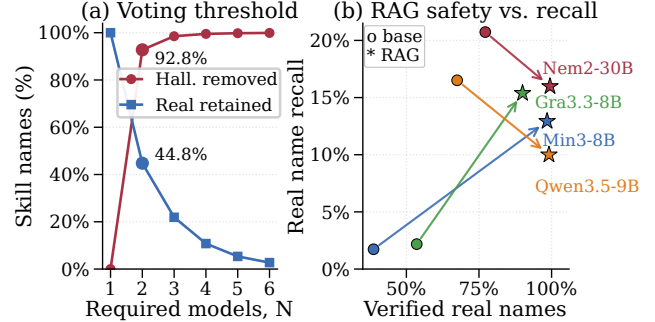
We next test whether the detection failure observed in RQ3 carries over to an end-to-end mitigation pipeline. In the self-refinement setup [32], the model first generates candidate skill names and then receives its own output in a second turn, where it is instructed to remove any name it cannot verify. This internal validation reduces the average hallucination rate from 40.8% to 33.2%, but the reduction comes mainly from suppressing outputs rather than from reliable verification. The no-output rate rises from 73.9% to 87.4% on average, and becomes extreme for Qwen 3.5 9B and Nemotron-Cascade-2 30B, reaching 97.5% and 96.7%, respectively. Therefore, the RQ3 detection failure is not merely an artifact of binary yes/no testing: when used as an operational defense, self-refinement trades hallucinations for no output and still leaves a high hallucination rate among the few names it retains.

Given the isolated outcomes of RAG and self-refinement, we also examined whether combining them could offer compounding benefits. Running self-refinement after RAG only marginally changes the average hallucination rate, from 3.2% under RAG to 2.7%, while increasing the no-output rate from 68.0% to 79.4%. Once RAG restricts the model to real skills, the extra self-refinement pass mostly removes additional outputs instead of adding a meaningful safety benefit.

**Finding 6.** Giving real names helps. Self-refinement is unreliable. RAG reduces hallucination rates sharply, from a 40.8% average down to 3.2%. Self-refinement alone is much weaker: it lowers the rate mostly by returning no output, because the second pass repeats the same verification weakness as the first.

Shifting away from single-model strategies, we evaluated a cross-model voting defense. Building on our insight from RQ2 that hallucinations are usually specific to one model while real skills are shared, we hypothesize that requiring multiple models to agree could act as a natural filter. Indeed, as shown in Figure 10(a), if we accept a name only when two independent models produce it, corresponding to  $N = 2$ , we eliminate 92.8% of hallucinations while keeping 44.8% of real skills. Raising the threshold to  $N = 3$  removes

over 98% of hallucinations, though real-skill preservation drops to 21.9%. This sharp drop confirms that hallucinated names are isolated to single models. This approach requires no external database or model updates, needing only multiple independent agents.


**Figure 10: Mitigation trade-offs for voting and RAG.**

While the aforementioned filters successfully suppress hallucinations, they introduce a severe penalty to the agent’s overall utility. Because a defense that simply returns no output is not a practical fix, we also measure *recall*, defined as the rate at which the agent successfully recommends the correct skill when a real one exists. Under RAG, safety and usefulness fundamentally trade off against each other. For the two weaker models, RAG improves recall by handing them correct answers they would not have guessed, with Ministral-3 rising from 1.7% to 12.9% and Granite 3.3 increasing from 2.2% to 15.4%. But for the two stronger models, recall still drops, as seen when Qwen 3.5 falls from 16.5% to 10.0% and Nemotron-Cascade-2 decreases from 20.7% to 16.0%, showing that retrieval constraints can suppress hallucinated names without reliably preserving the exact correct skill. Ultimately, even under RAG, no model recommends the correct skill more than 16.0% of the time. Figure 10(b) shows that RAG shifts outputs toward verified real names without making real-name recall reliably high. The agent becomes safer mainly by returning fewer hallucinated names, not by becoming reliably correct.

**Finding 7.** The strongest defense buys safety with no output. Requiring two independent models to agree removes most hallucinations, but it also drops many real skills. Retrieval shows the same tradeoff, as the agent becomes safer mainly by answering less, not by naming the right skill reliably.

## 5 Discussion and Limitations

**Discussion.** Our hallucination rates of 6.5% to 62.0% exceed the 5.2% to 21.7% reported for traditional packages [51]. This gap comes from a lack of knowledge about skills. The Agent Skills standard appeared in December 2025 [4], after the training cutoff of most models we test, so models often fall back on familiar package names from ecosystems such as npm [50] and PyPI [49]. As a result, a name like `eslint-config-prettier` is returned as a skill (Section 4.2), making the same hallucinated name useful for attacks across multiple ecosystems.

This explains the inadequacy of simple defenses. Web search verifies a tool’s existence, but not its validity as a skill. Self-refinement has the same problem because the second pass evaluates the answer using the same prior that produced the hallucination. RQ4 shows the cost of suppressing these names. Even the agent with the strongest defense returns the correct skill at most one time in six, while several configurations reduce hallucination mainly by returning no output. This trades false positives for false negatives, producing safer recommendations but fewer useful ones. The core challenge is therefore not silence, but retrieval of the right skill without admitting hallucinated skill names.

This trade-off also suggests that skill registries need defenses that are more explicit than package registries. Because a skill is primarily a natural-language instruction bundle, artifact-level checks provide weaker signals than they do for ordinary packages. A practical defense should therefore treat recommendation as a registry lookup. The agent should retrieve from an authenticated skill index and refuse names that cannot be resolved to a concrete SKILL.md. Namespace reservation for recurring hallucinated names may reduce ahead-of-time squatting, but it cannot replace exact lookup because our results show a long tail of model-specific names.

**Limitations.** Our study measures the attack *precondition*, in which agents hallucinate reusable skill names and defenses often miss them. We also verify the registration and installation path with a benign placeholder skill in an isolated sandbox, but we omit a full malicious compromise. Ethically, we did not register malicious skills, attack any public registry, or expose real users to the workflow. The exploitation step itself follows a known mechanism. Once a poisoned SKILL.md is installed, it enters the agent context and can act as instruction level prompt injection, as shown in prior work [19, 20, 29]. We establish this delivery channel is plausible because attackers can predict attractive names and register them in advance. Measuring whether real users install such recommendations requires a controlled user study. Finally, our recall analysis covers only the four open weight models used for mitigation, so we defer the full false-negative study across all 12 configurations.

## 6 Related Work

### 6.1 Supply Chain Attacks and Package Hallucination

While traditional supply chain attacks [15, 26, 34, 35] exploit mistyped legitimate packages, LLMs introduce novel threats via hallucinated recommendations. Following discoveries of package hallucinations [51], recent research categorized code generation errors [56] and proposed API-level mitigations [22]. However, extending these insights to the Agent Skills ecosystem demands a fundamental threat model shift. Unlike mature code registries supporting artifact comparison against established references, skill hallucinations lack legitimate counterparts. Furthermore, skills execute natural language instead of compiled code, rendering static analysis ineffective.

Our work is closest to package hallucination, but differs in both the object being named and the user’s trust path. A hallucinated package must usually survive dependency resolution, build tooling, and code review before it affects an application. A hallucinated skill can be installed because the same agent that recommends it also helps fetch and activate it. The payload is also not limited to

executable source code. Adversarial instructions can be embedded directly in a SKILL.md, where they become part of the agent’s future context. We therefore study not only whether names are false, but whether false names are persistent, reusable, and attractive enough for attackers to register before victims ask for them.

### 6.2 Agent Security and Prompt Injection

Installed malicious skills inherently threaten the execution integrity of LLM agents. Recent studies have surveyed widespread vulnerabilities in agent applications [3, 21, 44, 46, 54], demonstrating how malicious modules or online interactions amplify risks like data exfiltration and impersonation [14, 24]. A critical consequence of such compromises is indirect prompt injection, extensively benchmarked in recent literature [13, 19, 29, 47, 55]. Contemporary defenses utilize structured query separation [9], preference optimization [10], execution isolation [52], and guardrail agents [53] to sanitize adversarial inputs during inference. Yet, these methods implicitly assume the malicious payload arrives alongside the user query. Skill name hallucination circumvents these boundaries entirely. By tricking the user into installing a poisoned skill, the adversary injects adversarial instructions directly into the persistent configuration of the agent, bypassing standard alignment and runtime protections [30, 42, 57]. This makes installation a persistent and reusable trust boundary.

### 6.3 Threats in the Agent Skills Ecosystem

Consequently, the Agent Skills ecosystem presents a unique and largely unaddressed attack surface. Liu et al. [31] identify widespread vulnerabilities within published skills, while Qu et al. [43] demonstrate that malicious logic embedded in skill documentation can successfully manipulate agent behavior. Both works assume the attacker compromises existing skills or publishes payloads hoping for manual discovery. Our research complements these studies by formalizing vulnerabilities at the recommendation phase. This shifts the attack earlier than prior work considers. We demonstrate that attackers do not need to wait for organic discovery, but rather can weaponize the persistent hallucinations of LLMs in advance to actively drive victims toward malicious skills.

## 7 Conclusion

We studied skill name hallucination as a supply-chain attack across 12 configurations and 15,000 prompts. Every one hallucinated, at 6.5–62.0% overall and 43.1% on real questions. The names are targetable: models repeat a name up to 7.8 in 10, 410 shared names cover 15.3% of hallucinations, and 851 are real PyPI or npm packages. They also evade detection: self-checks match chance, and names sit a median of six edits from any real skill. RAG cuts hallucination by 37.6 percentage points, yet the best-defended agent recommends the right skill at most one in six. It is therefore a real, low-cost attack surface, and closing it requires registry-level reservation and verified recommendation, not tuning alone.

## 8 Data Availability

We release all prompts, skills, and responses at <https://figshare.com/s/4e408bce9b2f64224f79>.

## References

- [1] OpenAI. 2026. Agent Skills in Codex. <https://developers.openai.com/codex/skills>. Accessed: 2026 Jun.
- [2] Agent Skills. 2026. Agent Skills Overview. <https://agentskills.io/home>. Accessed: 2026 Apr.
- [3] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, et al. 2025. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, Vol. 2025. 79185–79220.
- [4] Anthropic. 2025. Introducing Agent Skills. <https://claude.com/blog/skills>. Accessed: 2026 Apr.
- [5] Anthropic. 2026. Agent Skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>. Accessed: 2026 Jun.
- [6] Anthropic. 2026. Anthropic’s Transparency Hub. <https://www.anthropic.com/transparency>. Accessed: 2026 Jun.
- [7] Anthropic. 2026. Claude Platform Release Notes. <https://platform.claude.com/docs/en/release-notes/overview>. Accessed: 2026 Apr.
- [8] Anthropic. [n. d.]. Agent Skills in the SDK. <https://platform.claude.com/docs/en/agent-sdk/skills>. Accessed: 2026 Jun.
- [9] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*. 2383–2400.
- [10] Sizhe Chen, Arman Zharmagambetov, Saeed Mahlouljifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2833–2847.
- [11] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AGENT-POISON: red-teaming LLM agents via poisoning memory or knowledge bases. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS ’24)*. Curran Associates Inc., Red Hook, NY, USA, Article 4136, 29 pages.
- [12] Cursor. 2026. Agent Skills. <https://cursor.com/docs/context/skills>. Accessed: 2026 Jun.
- [13] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *Advances in Neural Information Processing Systems* 37 (2024), 82895–82920.
- [14] Tian Dong, Minhui Xue, Guoxing Chen, Rayne Holland, Yan Meng, Shaofeng Li, Zhen Liu, and Haojin Zhu. 2025. The Philosopher’s Stone: Trojaning Plugins of Large Language Models. In *32nd Annual Network and Distributed System Security Symposium (NDSS)*.
- [15] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21–25, 2021*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/towards-measuring-supply-chain-attacks-on-package-managers-for-interpreted-languages/>
- [16] Gartner. 2025. Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>.
- [17] GitHub. 2025. GitHub Copilot Now Supports Agent Skills. <https://github.blog/changelog/2025-12-18-github-copilot-now-supports-agent-skills/>. Accessed: 2026 Jun.
- [18] Google. 2026. Google Antigravity Skills Documentation. <https://antigravity.google/docs/skills>. Accessed: 2026 Jun.
- [19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (Copenhagen, Denmark) (AISec ’23)*. Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985
- [20] Wenbo Guo, Wei Zeng, Chengwei Liu, Xiaojun Jia, Yijia Xu, Lei Tang, Yong Fang, and Yang Liu. 2026. MalSkillBench: A Runtime-Verified Benchmark of Malicious Agent Skills. [arXiv:2606.07131 \[cs.CR\]](https://arxiv.org/abs/2606.07131) <https://arxiv.org/abs/2606.07131>
- [21] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. 2025. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. *ACM Comput. Surv.* 58, 6, Article 162 (Dec. 2025), 36 pages. doi:10.1145/3773080
- [22] Nihal Jain, Robert Kwiatkowski, Baishakhi Ray, Murali Krishna Ramanathan, and Varun Kumar. 2025. On Mitigating Code LLM Hallucinations with API Documentation. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 237–248. doi:10.1109/ICSE-SEIP66354.2025.00027
- [23] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* 55, 12, Article 248 (March 2023), 38 pages. doi:10.1145/3571730
- [24] Hanna Kim, Minkyoo Song, Seung Ho Na, Seungwon Shin, and Kimin Lee. 2025. When LLMs go online: the emerging threat of web-enabled LLMs. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC ’25)*. USENIX Association, USA, Article 90, 20 pages.
- [25] Jan H. Klemmer, Stefan Albert Horstmann, Nikhil Patnaik, Cordelia Ludden, Cordell Burton, Carson Powers, Fabio Massacci, Akond Rahman, Daniel Votipka, Heather Richter Lipford, Awais Rashid, Alena Naiakshina, and Sascha Fahl. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS ’24)*. ACM, 2726–2740. doi:10.1145/3658644.3690283
- [26] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. 1509–1526. doi:10.1109/SP46215.2023.10179304
- [27] Jasmine Latendresse, SayedHassan Khatounabadi, Ahmad Abdellatif, and Emad Shihab. 2024. Is ChatGPT a Good Software Librarian? An Exploratory Study on the Use of ChatGPT for Software Library Recommendations. [arXiv:2408.05128 \[cs.SE\]](https://arxiv.org/abs/2408.05128) <https://arxiv.org/abs/2408.05128>
- [28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS ’20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [29] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1831–1847. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [30] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. Datasentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2190–2208.
- [31] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. [arXiv:2601.10338 \[cs.CR\]](https://arxiv.org/abs/2601.10338) <https://arxiv.org/abs/2601.10338>
- [32] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. SELF-REFINE: iterative refinement with self-feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS ’23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2019, 61 pages.
- [33] Microsoft. 2026. Use Agent Skills in VS Code. <https://code.visualstudio.com/docs/agent-customization/agent-skills>. Accessed: 2026 Jun.
- [34] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. 2023. Beyond typosquatting: an in-depth look at package confusion. In *32nd USENIX security symposium (USENIX security 23)*. 3439–3456.
- [35] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings (Lisbon, Portugal)*. Springer-Verlag, Berlin, Heidelberg, 23–43. doi:10.1007/978-3-030-52683-2\_2
- [36] OpenAI. 2023. Function calling and other API updates. <https://openai.com/index/function-calling-and-other-api-updates/>. Accessed: 2026 Apr.
- [37] OpenAI. 2026. Skills Catalog for Codex. <https://github.com/openai/skills>. Accessed: 2026 May.
- [38] OpenAI Developers. [n. d.]. Skills. <https://developers.openai.com/api/docs/guide/s/tools-skills>. Accessed: 2026 Jun.
- [39] OpenCode. 2026. Agent Skills. <https://opencode.ai/docs/skills>. Accessed: 2026 Jun.
- [40] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. *Commun. ACM* 68, 2 (Jan. 2025), 96–105. doi:10.1145/3610721
- [41] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS ’23)*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157
- [42] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. 2024. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *International Conference on Learning Representations*, Vol. 2024. 30988–31043.

- [43] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. arXiv:2604.03081 [cs.CR]. <https://arxiv.org/abs/2604.03081>
- [44] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitit, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=GEcwtMk1uA>
- [45] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2205–2222.
- [46] Zhuoxiang Shen, Jiarun Dai, Yuan Zhang, and Min Yang. 2025. Security Debt in LLM Agent Applications: A Measurement Study of Vulnerabilities and Mitigation Trade-offs. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, 559–570. doi:10.1109/ASE63991.2025.00053
- [47] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2026. Prompt Injection Attack to Tool Selection in LLM Agents. In *33rd Annual Network and Distributed System Security Symposium (NDSS)*.
- [48] SkillsMP. [n. d.]. Agent Skills Marketplace: Discover Claude Skills, Claude Code skills, Codex skills, and ChatGPT agent skills built around the open SKILL.md format. <https://skillsmp.com>. Accessed: 2026 May.
- [49] SONAR. 2026. Libraries.io. <https://libraries.io/pypi>. Accessed: 2026 Apr.
- [50] Sonatype. 2026. 2026 State of the Software Supply Chain. <https://www.sonatype.com/state-of-the-software-supply-chain/2026/software-infrastructure-growth>. Accessed: 2026 May.
- [51] Joseph Spracklen, Raveen Wijewickrama, AHM Nazmus Sakib, Anindya Maiti, and Bimal Viswanath. 2025. We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*. 3687–3706.
- [52] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *32nd Annual Network and Distributed System Security Symposium (NDSS)*.
- [53] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Nathaniel D. Bastian, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *ICML 2025 Workshop on Computer Use Agents*. <https://openreview.net/forum?id=ITuuEaXcSB>
- [54] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 1467–1490. doi:10.18653/v1/2024.findings-emnlp.79
- [55] Qiushi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624
- [56] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjuan Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA022 (June 2025), 23 pages. doi:10.1145/3728894
- [57] Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Jing Jiang, and Min Lin. 2024. Improved few-shot jailbreaking can circumvent aligned language models and their defenses. *Advances in neural information processing systems* 37 (2024), 32856–32887.
- [58] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: knowledge corruption attacks to retrieval-augmented generation of large language models. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC ’25)*. USENIX Association, USA, Article 197, 18 pages.